

计算方法B

12.25应交上机作业

宋思逸

2400010816

2025 年 12 月 22 日

1 梯度型方法的比较

1.1 问题描述

对下面的问题，比较最速下降法，BB方法和共轭梯度方法的有效性：

在三维Laplace问题的基础上，Fletcher设计了非二次函数

$$\frac{1}{2}x^T Gx - b^T x + \frac{1}{4}h^2 \sum_{ijk} x_{ijk}^4 \quad (1)$$

其中G与b的定义与本章数值实验中的问题一致，即：

$$T = \begin{bmatrix} 6 & -1 & 0 & \cdots & 0 \\ -1 & 6 & -1 & \ddots & \vdots \\ 0 & -1 & 6 & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & -1 \\ 0 & \cdots & 0 & -1 & 6 \end{bmatrix}, W = \begin{bmatrix} T & -I & 0 & \cdots & 0 \\ -I & T & -I & \ddots & \vdots \\ 0 & -I & T & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & -I \\ 0 & \cdots & 0 & -I & T \end{bmatrix} \quad (2)$$

$$G = \begin{bmatrix} W & -I & 0 & \cdots & 0 \\ -I & W & -I & \ddots & 0 \\ 0 & -I & W & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & -I \\ 0 & \cdots & 0 & -I & W \end{bmatrix} \quad (3)$$

且若该问题的解 x^* 已知，由 $b = Gx^*$ 可以计算出 b ，这里取 x^* 的分量为函数

$$x(u, v, w) = u(u-1)v(v-1)w(w-1) \cdot \exp\left\{-\frac{\sigma^2((u-\alpha)^2 + (v-\beta)^2 + (w-\gamma)^2)}{2}\right\} \quad (4)$$

$h = \frac{1}{m+1}$, x_{ijk} 为单位立方体上相应格点的值。

初始点可选为 $x_0 = 0$ ，问题中的参数 $\sigma, \alpha, \beta, \gamma$ 可取为 $\sigma = 20, \alpha = \beta = \gamma = 0.5$ 可选问题的规模为 m^3 其中 $m = 40$

说明：给出每种方法迭代所需的函数调用次数、梯度调用次数和CPU时间，并进行分析比较。

1.2 结果分析

=== Results ===

Steepest Descent (Armijo)

```

iters          : 1896
f(x)           : -2.008177096839334e-03
||g||          : 9.992507e-07
f calls        : 6606
g calls        : 1897
cpu time (s)   : 30.811439

```

BB (BB1 + Armijo safeguard)

```

iters          : 187
f(x)           : -2.008177080545059e-03
||g||          : 9.851591e-07
f calls        : 376
g calls        : 188
cpu time (s)   : 2.031372

```

Nonlinear CG (PR+ + strong Wolfe)

```

iters          : 164
f(x)           : -2.008177101586940e-03
||g||          : 9.982842e-07
f calls        : 901
g calls        : 533
cpu time (s)   : 5.863980

```

通用字段（3 个方法都有） 1) iters 含义：外层迭代次数 (k)（更新一次 $x_{k+1} = x_k + \alpha_k d_k$ 算 1 次迭代）。

SD: 1896 次

BB: 187 次

NLCG: 164 次

说明 SD 走得最慢；BB、NLCG 用更“聪明”的步长/方向，所以迭代显著少。

2) $f(x)$ 含义：算法结束时的目标函数值 $f(x) = \frac{1}{2}x^T Gx - b^T x + \frac{1}{4}h^2 \sum_{ijk} x_{ijk}^4$

三个方法的 $f(x)$ 都在 $-2.00817709 \times 10^{-3}$ 附近，说明它们收敛到了几乎同一个最小点/同一水平（差异主要来自停止阈值和数值误差）。

3) $\|g\|$ 含义：终止点的梯度范数（通常是 2-范数） $\|g(x)\|_2, g(x) = Gx - b + h^2 x^3$, $\text{tol} = 1e-6$ ，所以当 $|g(x)| \leq 10^{-6}$ 就认为“足够接近驻点/最优解”。三种方法最后都在 10^{-6} 量级，说明都满足了停止条件。

4) f calls 含义：目标函数 $f(\cdot)$ 被计算的次数（包括线搜索里每次试探步长 α 时对 $f(x + \alpha d)$ 的计算）。它反映了：线搜索“试了多少次”、以及算法对 f 的依赖程度。

SD: 6606 次（很大，主要因为每步 Armijo 回溯要反复试 α ）

BB: 376 次（非常少，说明 BB 的初始步长通常就比较合适，很少回溯）

NLCG: 901 次（中等，强 Wolfe 线搜索一般要比 Armijo 更“费函数值”）注意：一次迭代不一定只调用 1 次 f 。线搜索会让 f -calls 远大于 iters 。

5) g calls 含义：梯度 $g(\cdot)$ 被计算的次数（同样包括线搜索中可能需要的梯度评估；强 Wolfe 通常需要多次梯度）。

SD: $1897 \approx \text{iters} + 1$ （基本每次迭代算一次梯度，很典型）

BB: $188 \approx \text{iters} + 1$ （同上）

NLCG: 533（明显大于 $\text{iters} + 1 = 165$ ，原因是强 Wolfe 线搜索里会反复算梯度来检查曲率条件 $|g(x + \alpha p)^T p|$ ）

6) cpu time (s) 含义：程序运行用时（秒）。

SD: 30.81 s（最慢）

BB: 2.03 s（最快）

NLCG: 5.86 s（居中）

时间大致和“算了多少次 Gop （从而多少次梯度/函数）+ 线搜索开销”相关。

算一次梯度需要一次 $\text{Gop}(x)$ ；

算一次函数也需要一次 $\text{Gop}(x)$

所以 f calls + g calls 的总量对时间影响很大：

SD: $6606 + 1897 = 8503$ 次（大） $\rightarrow$ 时间大

BB: $376 + 188 = 564$ 次（小） $\rightarrow$ 时间小

NLCG: $901+533=1434$ 次 (中) $\rightarrow$ 时间中

1.3 小结

最速下降法: 迭代多、函数调用多、耗时最大, 但最终也能达到同等精度。

BB 方法: 迭代次数大幅减少, 函数/梯度调用极少, CPU 时间最短, 整体最有效。

非线性共轭梯度 (PR+): 迭代也少, 但强 Wolfe 线搜索导致梯度/函数调用更多, 因此时间介于 BB 和 SD 之间; 收敛效果同样达到容差。

2 共轭梯度方法的比较

2.1 问题描述

对非线性大规模最优化问题, 进行PRP,FR,PRP⁺,CD及DY五种方法有效性的比较。其中目标函数Convex 1函数

$$f(x) = \sum_{i=1}^n (e^{x_i} - x_i) \quad (5)$$

其中问题的规模选为 $n=100$, 选取 $x_0 = (\frac{1}{n}, \dots, \frac{i}{n}, \dots, 1)$ 。给出每种方法迭代所需的函数调用次数、梯度调用次数和CPU时间。

2.2 运行结果

```
=== Results (Convex1) ===

FR
iters      : 12
f(x)       : 1.0000000000000001e+02
||g||      : 3.972848e-07
f calls    : 13
g calls     : 13
cpu time (s) : 0.017949

PRP
iters      : 9
f(x)       : 1.0000000000000000e+02
||g||      : 1.264569e-08
```

```

f calls      : 10
g calls      : 10
cpu time (s) : 0.002112

PRP+
iters        : 5
f(x)         : 1.0000000000000000e+02
||g||        : 2.715392e-08
f calls      : 6
g calls      : 6
cpu time (s) : 0.000888

CD
iters        : 17
f(x)         : 1.0000000000000004e+02
||g||        : 9.384195e-07
f calls      : 18
g calls      : 18
cpu time (s) : 0.001102

DY
iters        : 22
f(x)         : 1.0000000000000003e+02
||g||        : 7.398039e-07
f calls      : 23
g calls      : 23
cpu time (s) : 0.001802

```

事实上，此问题有精确解

$$\nabla f(x) = e^x - 1 = 0 \implies x^* = 0 \quad (6)$$

$$f(x^*) = \sum_{i=1}^n (e^0 - 0) = 100 \quad (7)$$

因此 $f(x)$ 基本都收敛到100. $\|g\|$ 很小说明都满足一阶必要条件，迭代次数其实都比较小，在此问题中 PRP^+ 最快， DY 最慢。总的来说五种方法都能快速且较精确解出该问题